

UNIVERSIDADE FEDERAL DE SANTA CATARINA

**Uma ferramenta para extração de esquemas de
bancos de dados NoSQL orientados a
documentos**

Felipe de Souza da Costa

Florianópolis

2017/2

Felipe de Souza da Costa

Uma ferramenta para extração de esquemas de bancos de dados NoSQL orientados a documentos

Trabalho de conclusão de curso apresentado como parte dos requisitos para obtenção do título de Bacharel, do curso de Sistemas de Informação na Universidade Federal de Santa Catarina.

Orientador: Prof^o Ronaldo dos Santos Mello, Dr.

Coorientador: Prof^o Angelo Augusto Frozza, M.Sc.

Florianópolis

2017/2

Felipe de Souza da Costa

Uma ferramenta para extração de esquemas de bancos de dados NoSQL orientados a documentos

Trabalho de conclusão de curso apresentado como parte dos requisitos para obtenção do título de Bacharel, do curso de Sistemas de Informação na Universidade Federal de Santa Catarina.

Profº Frank Augusto Siqueira, Dr.
Coordenador do Curso

Banca Examinadora:

Profº Ronaldo dos Santos Mello, Dr.
Orientador
Universidade Federal de Santa Catarina

Profº Angelo Augusto Frozza, M.Sc.
Coorientador
Instituto Federal Catarinense

Profª Carina Friedrich Dorneles, Drª
Universidade Federal de Santa Catarina

Profº Roberto Willrich, Dr.
Universidade Federal de Santa Catarina

Florianópolis

2017/2

RESUMO

Os bancos de dados NoSQL estão se tornando cada vez mais populares no desenvolvimento de aplicações, devido à sua capacidade de lidar com grandes volumes de dados e, também, pela ausência de um esquema de dados explícito. Embora a maioria dos bancos de dados NoSQL não tenha esquema, as informações sobre as propriedades estruturais dos dados persistidos são essenciais durante o desenvolvimento de aplicações. Sem o conhecimento dessas propriedades estruturais, atividades de desenvolvimento de aplicações ou análise de dados tornam-se um trabalho custoso e, às vezes, impraticável. Sendo assim, o presente trabalho propõe o desenvolvimento de uma ferramenta que, dada uma coleção de dados armazenados em um banco de dados NoSQL orientado a documentos, realize a extração de seu esquema, com o objetivo de facilitar o desenvolvimento, integração e validação de dados, bem como a análise dos mesmos. Espera-se, com este trabalho, a elaboração de um documento contendo o relato sobre os estudos realizados e os resultados dos testes com a ferramenta implementada, além da publicação do código fonte da mesma.

Palavras-chaves: Extração de esquemas, NoSQL, Banco de Dados orientado a documentos, JSON, JSON Schema.

LISTA DE ILUSTRAÇÕES

Figura 1 – Os 5 Vs de <i>Big Data</i>	20
Figura 2 – Teorema CAP	22
Figura 3 – Modelos de dados de banco de dados NoSQL	24
Figura 4 – Representação de um objeto em JSON	28
Figura 5 – Representação de um <i>array</i> em JSON	28
Figura 6 – Representação de um valor em JSON	29
Figura 7 – Grafo de Identificação de Estrutura proposto por Klettke, Störl e Scherzinger (2015)	32
Figura 8 – Etapas da Extração de esquemas proposta por Klettke, Störl e Scherzinger (2015)	33
Figura 9 – Exemplo de uma coleção de objetos JSON	34
Figura 10 – Etapas da Extração de esquemas proposta por Ruiz, Morales e Molina (2015)	35
Figura 11 – Exemplo de esquema bruto (<i>raw schema</i>)	36
Figura 12 – Metamodelo NoSQL <i>Schema</i> para representar um esquema de banco de dados NoSQL orientado a agregados	36
Figura 13 – Exemplo de um esquema NoSQL	37
Figura 14 – <i>Framework</i> para gerenciamento de esquemas.	38
Figura 15 – Arquitetura da ferramenta proposta	42
Figura 16 – Exemplo de um documento JSON	43
Figura 17 – Exemplo de um documento armazenado no <i>MongoDB</i>	44
Figura 18 – Exemplo do <i>Raw Schema</i> gerado	45

LISTA DE ABREVIATURAS E SIGLAS

ACID	<i>Atomicity, Consistency, Isolation, Durability</i>
API	<i>Application Programming Interface</i>
BASE	<i>Basically Available, Soft state, Eventual consistency</i>
BD	Banco de Dados
BLOB	<i>Binary Large Object</i>
BSON	<i>Binary JSON</i>
CLI	<i>Command Line Interface</i>
DTD	<i>Document Type Definition</i>
GUI	<i>Graphical User Interface</i>
HTTP	<i>Hypertext Transfer Protocol</i>
JSON	<i>JavaScript Object Notation</i>
MVC	<i>Model, View, Controller</i>
NoSQL	<i>Not SQL ou Not Only SQL</i>
SGBDR	Sistema de Gerenciamento de Banco de Dados Relacional
TCC	Trabalho de Conclusão de Curso
UML	<i>Unified Modeling Language</i>
XML	<i>eXtensible Markup Language</i>

SUMÁRIO

1	Introdução	13
1.1	Motivação	15
1.2	Objetivos	15
1.2.1	Objetivo Geral	15
1.2.2	Objetivos Específicos	15
1.3	Justificativa	16
1.4	Metodologia	16
1.5	Estrutura do trabalho	17
2	Fundamentação Teórica	19
2.1	<i>Big Data</i>	19
2.2	Teorema CAP	21
2.3	NoSQL	22
2.3.1	Modelo de dados	23
2.3.2	Modelo de dados orientados a agregados	24
2.3.3	Banco de dados chave-valor	24
2.3.4	Banco de dados colunar	25
2.3.5	Banco de dados orientado a grafos	26
2.3.6	Banco de dados orientados a documentos	26
2.3.6.1	<i>MongoDB</i>	27
2.4	JSON	27
2.5	JSON <i>Schema</i>	29
3	Trabalhos Relacionados	31
3.1	<i>Schema Extraction and Structural Outlier Detection for JSON-based NoSQL Data Stores</i>	31
3.2	<i>Inferring Versioned Schemas from NoSQL Databases and its Applications</i>	34
3.3	<i>Schema Management for Document Stores</i>	38
4	Andamento do Trabalho	41

4.1	Arquitetura	41
4.2	Processo de extração de esquemas	42
5	Atividades Futuras	47
	Referências	49

1 INTRODUÇÃO

A explosão do mercado de *Big Data* fez com que grandes empresas demandassem por bancos de dados (BDs) capazes de armazenar e processar grandes volumes de dados de forma eficaz com um alto desempenho em operações de leitura e escrita. Sendo assim, os BDs relacionais — ditos BDs tradicionais — começaram a enfrentar vários desafios quanto a alta concorrência em operações de leitura e escrita, armazenamento de *Big Data* de forma eficiente, capacidade de suportar expansões horizontais e *upgrades*, garantir um serviço rápido e ininterrupto, além de ter menores custos de gestão e de operação (HAN et al., 2011).

Para resolver essas necessidades, uma variedade de sistemas de BDs surgiu, com o propósito de facilitar a expansão, apoiar o armazenamento em massa, suportar eficientemente operações de leitura e escrita e, além de tudo, ter um baixo custo de operação e manutenção. Para atender a essas necessidades, esses BDs perdem as propriedades ACID (*Atomicity, Consistency, Isolation, Durability*) dos BDs relacionais em troca de manter as propriedades BASE (*Basically Available, Soft state, Eventual consistency*) (TUDORICA; BUCUR, 2011).

A esses BDs é dado o nome de BD NoSQL — conceito definido em 1998 por Carlo Strozzi e reintroduzido em 2009, por Eric Evans, em um evento realizado em São Francisco (EUA), sobre BDs não relacionais, distribuídos e de código aberto (SADALAGE; FOWLER, 2012).

O termo NoSQL é um acrônimo para “*Not Only SQL*” ou “*Not SQL*”, uma definição que não está totalmente acordada no meio acadêmico. (CATTELL, 2011; SADALAGE; FOWLER, 2012; RUIZ; MORALES; MOLINA, 2015)

No contexto deste trabalho, o termo NoSQL é utilizado no sentido de “*Not SQL*”, pois interpretar um BD como “*Not Only SQL*” pode tornar o termo sem sentido, já que um BD NoSQL não é de fato um BD SQL. O termo “*Not Only SQL*” faz mais sentido ao se considerar o ecossistema de BDs, porém, para isso já existe outra *buzzword*¹: Persistência Poliglota (SADALAGE; FOWLER, 2012).

¹ Uma palavra ou frase que está na moda em um momento específico ou em um contexto particular.

Uma característica muito importante de BDs NoSQL é o fato serem *schemaless*, ou seja, permitirem o armazenamento de dados sem o conhecimento prévio da estrutura dos mesmos (SADALAGE; FOWLER, 2012; RUIZ; MORALES; MOLINA, 2015).

O modelo de dados de BDs tradicionais é, principalmente, o modelo relacional, suportando operações de associação de tabelas e transações ACID. No entanto, em NoSQL, não há um único modelo de dados padrão a ser utilizado. Cada solução adota um modelo de dados diferente e os mesmos podem ser classificados em quatro categorias amplamente utilizadas: chave-valor, documento, família de colunas e grafos.

Os BDs NoSQL orientados a documentos, por exemplo, armazenam e recuperam documentos, os quais podem ser nos formatos XML² (*eXtensible Markup Language*), JSON³ (*JavaScript Object Notation*) ou BSON⁴ (*Binary JSON*) (HASHEM; RANC, 2016; INDRAWAN-SANTIAGO, 2012; HAN et al., 2011). Em geral, não possuem esquema, ou seja, os documentos não precisam possuir uma estrutura em comum. Os principais BDs dessa categoria são o *MongoDB*⁵ e o *CouchDB*⁶.

Em BDs relacionais, antes de armazenar dados é preciso ter uma estrutura previamente definida — que indica quais tabelas e colunas existem e quais tipos de dados cada coluna pode armazenar — ou seja, um esquema. No contexto NoSQL, armazenar dados é muito mais informal, pois os BDs NoSQL permitem que sejam armazenados quaisquer dados sobre uma chave ou uma coluna escolhida. Além disso, permitem que sejam adicionados, livremente, campos aos registros do banco de dados, sem ter de definir primeiro quaisquer mudanças na estrutura, como ocorre nos BDs relacionais (SADALAGE; FOWLER, 2012). A ausência de esquema evita o problema de registros com vários elementos sem valor, quando estes não são uniformes (cada registro possui um conjunto diferente de dados), permitindo que cada registro contenha apenas o necessário — nada a mais, nada a menos.

No entanto, a ausência de um esquema explícito, não acarreta a ausência total de esquema, visto que sempre há um esquema implícito nos dados e em aplicações de BDs (RUIZ; MORALES; MOLINA, 2015).

² <https://www.w3.org/XML/>

³ <http://json.org/>

⁴ <http://bsonspec.org/>

⁵ <https://www.mongodb.com/>

⁶ <http://couchdb.apache.org/>

1.1 Motivação

A flexibilidade de esquema é uma força motriz por trás da popularidade dos BDs NoSQL ([SADALAGE; FOWLER, 2012](#)). Embora essa flexibilidade de esquema pareça ser um ponto positivo, a informação sobre o esquema é essencial, por exemplo, durante o desenvolvimento de uma aplicação. Caso contrário, o acesso aos dados torna-se mais oneroso.

Na maioria das vezes, as informações de esquema de BDs NoSQL estão implícitas no código da aplicação, o que pode se tornar um grande problema se múltiplos aplicativos, desenvolvidos por pessoas diferentes, acessarem o mesmo BD ([SADALAGE; FOWLER, 2012](#)). Sendo assim, é preciso ter alguma noção confiável de sua estrutura, pois sem o seu conhecimento, qualquer desenvolvimento de aplicações ou análise significativa de dados, torna-se um trabalho custoso e, às vezes, impraticável ([KLETTKE; STÖRL; SCHERZINGER, 2015](#)).

Desta motivação, surge o propósito do presente Trabalho de Conclusão de Curso (TCC), o desenvolvimento de uma ferramenta que, dada uma coleção de documentos no formato JSON, armazenados em um BD NoSQL orientado a documentos, extraía o seu esquema implícito na coleção para facilitar o desenvolvimento, integração e validação de dados, bem como a análise dos mesmos.

1.2 Objetivos

1.2.1 Objetivo Geral

O presente TCC tem por objetivo geral o desenvolvimento de uma ferramenta que possibilite a extração de esquema de uma coleção de documentos JSON, persistidos em um BD NoSQL orientado a documentos.

1.2.2 Objetivos Específicos

Os objetivos específicos necessários para se alcançar o objetivo geral são os seguintes:

- Extrair os esquemas brutos (*raw schema*) de documentos JSON armazenados em

BD NoSQL orientados a documentos;

- Mapear os esquemas brutos (*raw schema*) para um único esquema no padrão JSON *Schema*;
- Avaliar e validar a ferramenta proposta através de um estudo de caso.

1.3 Justificativa

Conhecer a estrutura de uma coleção de dados armazenados em um BD NoSQL ou seja, o esquema, é essencial durante o desenvolvimento de aplicações. No entanto, ainda existem poucas ferramentas no mercado para esse propósito e, normalmente, as mesmas não adotam um formato padrão para representação do esquema gerado.

Sendo assim, a ferramenta proposta nesse TCC tem como objetivo gerar um esquema único de uma coleção de documentos no formato JSON (formato padrão para intercâmbio de dados complexos e heterogêneos ([IETF, 2014](#))), armazenados em um BD NoSQL orientado a documentos. O esquema gerado segue a definição JSON *Schema* — a definição está atualmente em seu sexto rascunho ([JSON Schema, 2017](#)), e está sendo estabelecido como um padrão para especificação de esquemas JSON — e pode auxiliar no desenvolvimento de aplicações, na integração e na validação de dados, bem como na análise de uma coleção de dados.

1.4 Metodologia

A primeira etapa do trabalho consistiu em um levantamento do estado da arte sobre “*extração de esquemas de BDs NoSQL*”. Para isto, foi feita a leitura de livros, artigos, *surveys* e publicações pertinentes ao assunto. A relevância das obras encontradas foi definida em conjunto com o orientador e coorientador. A partir desse levantamento, foi feito um estudo teórico sobre as técnicas e tecnologias envolvidas no processo de extração de esquemas de BDs NoSQL.

A segunda etapa focou na análise dos trabalhos relacionados ao tema abordado, para fazer o levantamento de requisitos e das características mínimas de ferramentas do gênero, com o objetivo de se propor uma nova ferramenta que ofereça um diferencial em relação às já existentes.

A terceira etapa, consiste no levantamento de requisitos, escolha das tecnologias que serão utilizadas, definição da arquitetura e o desenvolvimento da ferramenta proposta.

Na quarta etapa, será realizado um estudo de caso e uma análise comparativa entre a ferramenta proposta e os trabalhos relacionados.

1.5 Estrutura do trabalho

O presente trabalho se divide em seis capítulos. Este primeiro apresenta os objetivos, a metodologia e o contexto de surgimento deste TCC. No capítulo seguinte é apresentada a revisão dos fundamentos teóricos que servem como base ao entendimento do desenvolvimento da ferramenta proposta. O terceiro capítulo reúne uma análise dos trabalhos relacionados à ferramenta proposta. O andamento atual do trabalho será descrito no quarto capítulo e as atividades futuras, no quinto capítulo.

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo apresenta uma revisão dos conceitos essenciais para o entendimento e desenvolvimento do presente trabalho. São explicados conceitos sobre *Big Data*, NoSQL, JSON e JSON *Schema*.

2.1 *Big Data*

Big Data é um *buzzword* utilizado para definir um grande volume de dados não estruturados — na faixa de *exabytes* (10^{18}) ou mais — que requer novas tecnologias e arquiteturas, para que seja possível seu processamento, gerenciamento e armazenamento de forma efetiva e eficiente (SADALAGE; FOWLER, 2012).

Grandes empresas, como a *Google* e a *Amazon*, lidaram com grandes volumes de dados e com um tráfego cada vez maior no início do século XX. Escalar verticalmente se tornou inviável, pois adquirir máquinas maiores — com maior poder de processamento e com maior capacidade de armazenamento — significava maiores custos, sem mencionar que há limites físicos quanto ao aumento do tamanho (HAN et al., 2011).

A solução para processar esse grande volume de dados foi escalar horizontalmente, ou seja, utilizar mais máquinas, com poder de processamento menor, de forma distribuída. No entanto, essas empresas se depararam com um novo problema: a incompatibilidade entre os BDs relacionais e *clusters*, pois os BDs relacionais não foram projetados para serem executados de forma distribuída, sendo assim, não suportavam facilmente a fragmentação de dados.

Essa incompatibilidade fez com que essas empresas desenvolvessem soluções próprias para o armazenamento de grandes volumes de dados. *BigTable*⁷ e *Dynamo*⁸ foram as soluções propostas pela *Google* e *Amazon*, respectivamente, com suporte a fragmentação e replicação de dados (SADALAGE; FOWLER, 2012).

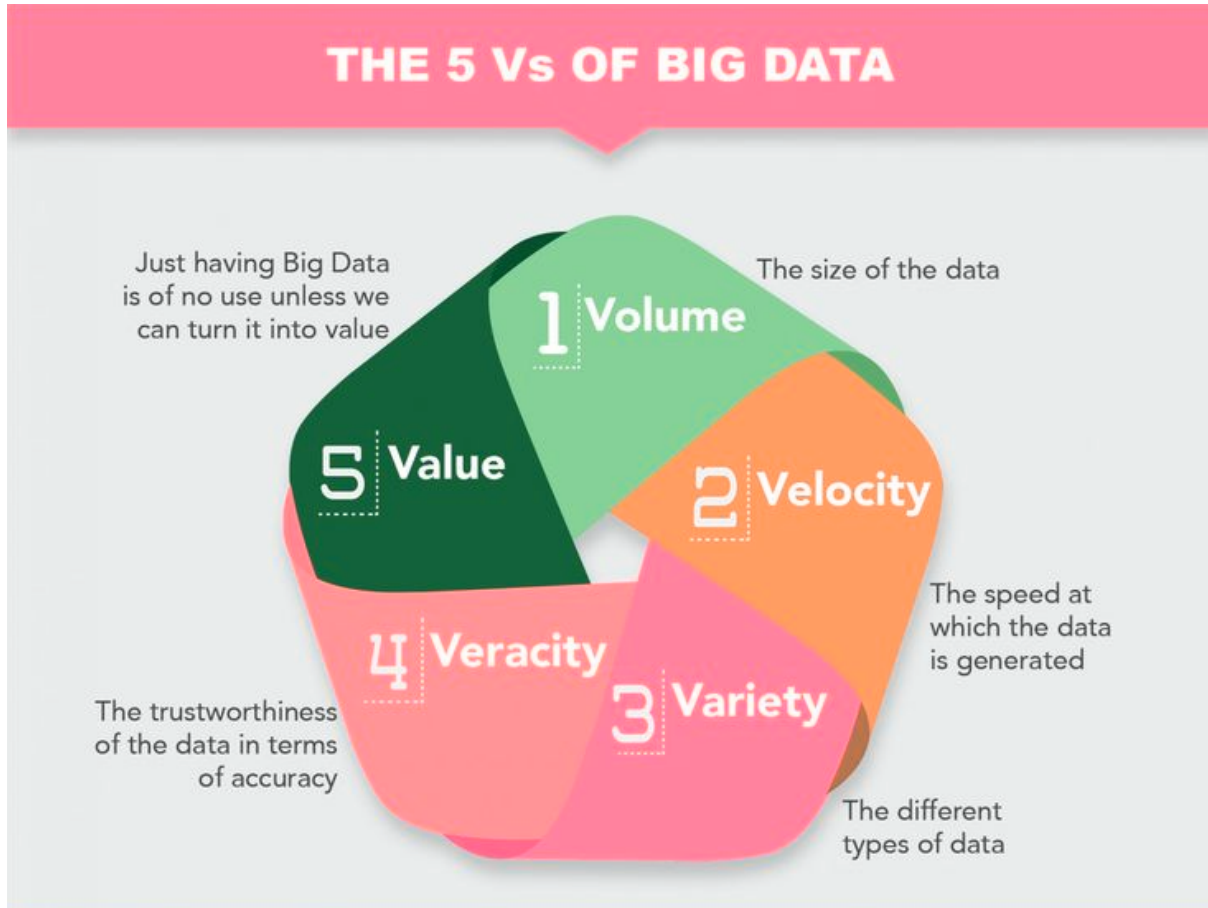
O volume de dados é apenas uma das propriedades de *Big Data*, outras propriedades são: variedade, velocidade, valor e veracidade. Essas propriedades são conhecidas

⁷ <https://cloud.google.com/bigtable/>

⁸ <https://aws.amazon.com/pt/dynamodb/>

como “os 5 Vs de Big Data”, conforme mostrado na figura 1 ([Big Data is the new black, 2016](#)).

Figura 1: Os 5 Vs de *Big Data*



Fonte: [Big Data is the new black \(2016\)](#)

Cada propriedade representada na figura 1 é brevemente expressa abaixo:

1. *Volume*: Refere-se ao volume de dados, que é força motriz por trás do conceito de *Big Data*, já visto anteriormente;
2. *Velocidade*: Refere-se à velocidade em que os dados são gerados, transmitidos e agregados de diversas fontes;
3. *Variedade*: Trata a variedade de estrutura dos dados, — dados estruturados, semiestruturados e não estruturados ([KATAL; WAZID; GOUDAR, 2013](#)) — pois os mesmos podem ser provenientes de diferentes fontes de dados, como páginas *web*, *logs*, redes sociais, *e-mail*, documentos, dados de sensores, entre outros ([SAGIROGLU; SINANC, 2013](#));

4. *Veracidade*: Está relacionado à autenticidade dos dados, pois com um grande volume de dados sendo gerados em alta velocidade, muitas informações podem ser irrelevantes. Sendo assim, é necessário realizar filtros e deles extrair o que realmente for relevante ([Big Data is the new black, 2016](#));
5. *Valor*: Refere-se ao valor gerado por *Big Data*, pois para garantir que os outros 4 Vs tragam rentabilidade, é essencial que os resultados de *Big Data* agreguem valor para a organização ([KATAL; WAZID; GOUDAR, 2013](#)).

Segundo [Kaisler et al. \(2013, p.1\)](#), um grande desafio para os pesquisadores e para os profissionais de TI é que a taxa de crescimento de *Big Data* ultrapassa rapidamente a capacidade de conceber sistemas adequados para lidarem com os dados de forma eficaz, bem como ultrapassam também a capacidade de analisar esses dados para extrair informações e conhecimentos úteis para a tomada de decisão.

2.2 Teorema CAP

Segundo [Brewer \(2000\)](#), um sistema distribuído tem três propriedades desejáveis:

- *Consistência* (C) – equivalente a ter uma única cópia atualizada dos dados;
- *Alta disponibilidade* (A) desses dados (para atualizações);
- *Tolerância* às Partições de rede (P) – sistema continua funcionando mesmo quando um problema ocorre na rede, dividindo o sistema em duas ou mais partições.

No entanto, [Brewer \(2000\)](#) afirma que um sistema distribuído só pode garantir duas entre essas três propriedades simultaneamente.

A figura 2 mostra as relações entre consistência, disponibilidade e tolerância à partição de rede. Por exemplo, um sistema baseado em CA, garante consistência e alta disponibilidade. Porém, não consegue ser tolerante à partições de rede. Essa característica é comum entre os BDs do modelo relacional. Um sistema baseado em PA garante alta disponibilidade e tolerância à partições de rede. No entanto, não consegue garantir consistência, pois um dado em um nodo qualquer pode não ser o dado mais atualizado. Um

sistema baseado em CP, garante consistência e tolerância à partições de rede. Porém, não consegue garantir alta disponibilidade, pois para manter a consistência, na atualização de um dado, é preciso bloquear o acesso ao mesmo até que a atualização do dado seja refletida (propagada) em todos os nós do sistema.

Figura 2: Teorema CAP



Fonte: [Brewer \(2000, p. 4\)](#)

Como em sistemas distribuídos, ser tolerante à partições de rede é essencial, escolher entre consistência e disponibilidade torna-se uma escolha não necessariamente binária. Sendo assim, deve-se otimizar a consistência e a disponibilidade para, então, conseguir algum *trade-off*⁹ das três propriedades ([BREWER, 2012](#)).

2.3 NoSQL

Os BDs NoSQL surgiram como uma solução aos problemas que grandes empresas enfrentaram no começo do século XX, quanto ao processamento e armazenamento de grandes volumes de dados.

⁹ Perder uma qualidade ou aspecto de algo, ganhando em troca outra qualidade ou aspecto.

Segundo Tudorica e Bucur (2011, p. 1), um BD NoSQL pode ser definido como sendo:

Um sistema de banco de dados que é distribuído, pode não exigir esquemas de tabela fixa, geralmente evitam operações de *join*, normalmente escalam horizontalmente, não expõe uma interface SQL e podem ser *open source*.

Ao oferecer alto desempenho e alta disponibilidade, esses BDs não priorizam a consistência, pois, de acordo com o teorema CAP (ver seção 2.2), alta disponibilidade e consistência, são propriedades que não são possíveis de serem garantidas de forma absoluta e simultânea, quando se opta por tolerância a partição de rede, que é o caso de sistemas distribuídos (BREWER, 2000).

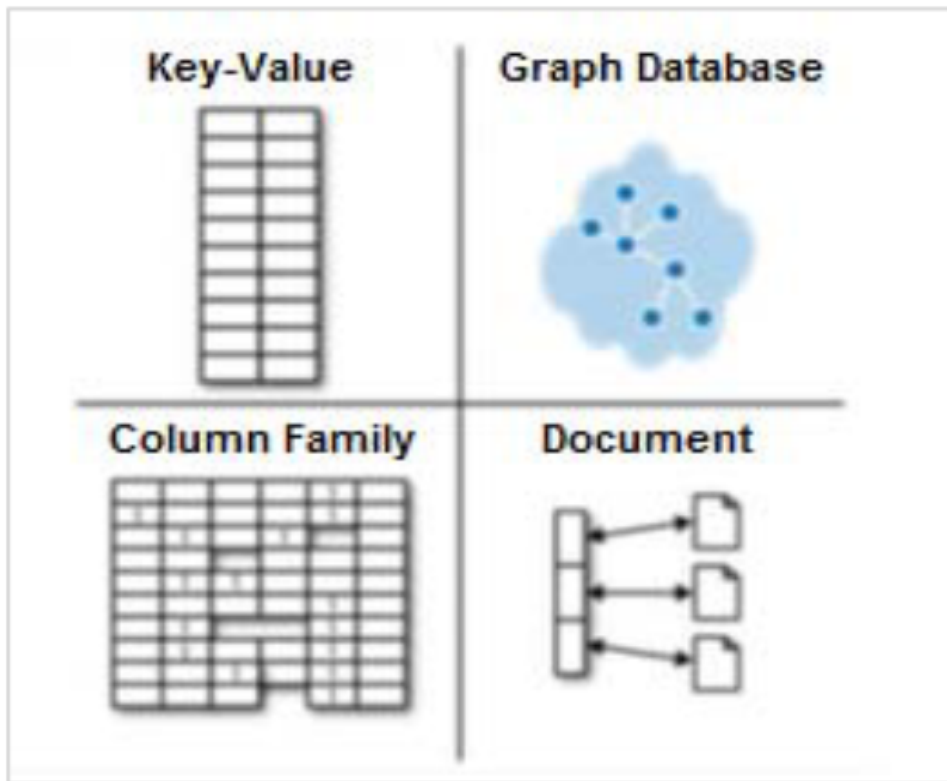
Por não priorizar a consistência, esses bancos de dados acabam perdendo as características ACID dos BDs relacionais, em troca de manter um BD BASE. Ao adotar as propriedades BASE, esses BDs garantem que as atualizações são eventualmente propagadas para todos os nós, mas que existem garantias limitadas sobre a consistência das leituras. Com isso, esses BDs conseguem garantir alto desempenho em operações de leitura e escrita, além de garantir alta disponibilidade e escalabilidade (SADALAGE; FOWLER, 2012).

2.3.1 Modelo de dados

O modelo relacional predominou por muitos anos como um modelo de dados padrão para BDs. Esse modelo pode ser descrito por um conjunto de tabelas, constituída por linhas e colunas. As linhas, também chamadas de tuplas, representam entidades de interesse e as colunas, por sua vez, representam as características (propriedades) das entidades. Uma coluna contém um único valor e este pode se referir a outra linha na mesma tabela ou em uma tabela diferente, caracterizando assim um relacionamento entre entidades (SADALAGE; FOWLER, 2012).

No contexto de BDs NoSQL, não há um único modelo de dados padrão utilizado. Cada solução adota um modelo de dados diferente e os mesmos podem ser classificados em quatro categorias amplamente utilizadas: chave-valor, documento, família de colunas e grafos, conforme representado na figura 3 (HASHEM; RANC, 2016).

Figura 3: Modelos de dados de banco de dados NoSQL



Fonte: [Hashem e Ranc \(2016, p. 1\)](#)

2.3.2 Modelo de dados orientados a agregados

No modelo relacional, as informações a serem armazenadas são divididas em um conjunto de tuplas. Uma tupla é uma estrutura de dados que contém um valor, não permitindo o aninhamento de outras tuplas (uma tupla dentro de outra) e nem de lista de valores.

A orientação agregada — utilizada pelos modelos de dados chave-valor, documento e família de colunas — os dados são tratados como unidades e tem uma estrutura mais complexa do que um conjunto de tuplas. Um agregado permite o aninhamento de listas e de outras estruturas de dados ([SADALAGE; FOWLER, 2012](#)).

2.3.3 Banco de dados chave-valor

Esses BDs são os menos complexos, pois todos os dados consistem, basicamente, em uma chave indexada e um valor. Conhecidos também como tabelas de *hash* distribuídas, permitem uma velocidade de consulta maior do que uma consulta em um

BD relacional (JMTAURO; S; A.B, 2012).

O valor, do par chave-valor, é um BLOB¹⁰ (*Binary Large Object*) que o BD apenas armazena, sem se preocupar ou saber o que há dentro dele (SADALAGE; FOWLER, 2012).

Além de suportar armazenamento em massa, os BDs chave-valor oferecem um alto desempenho em operações de leitura e escrita concorrentes. Alguns dos BDs chave-valor mais populares são *Riak*¹¹, *Redis*¹², *MemcachedDB*¹³, *BerkeleyDB*¹⁴, *Amazon DynamoDB*¹⁵ e *Voldemort*¹⁶.

2.3.4 Banco de dados colunar

Os BDs colunares, conhecidos também como BD de família de colunas, oferecem um alto desempenho e uma arquitetura altamente escalável.

Esses BDs podem ser considerados como um tipo específico de BD chave-valor. Porém, são mais complexos e, neste caso, muda-se o paradigma de orientação a registros ou *tuplas*, como utilizado no modelo relacional, para orientação a atributos ou colunas.

BDs colunares utilizam, como modelo de dados, uma tabela com um conjunto predefinido de colunas, não suportando associação entre tabelas, como ocorre no modelo relacional (HAN et al., 2011). As tabelas são constituídas por colunas, famílias de colunas e supercolunas.

A coluna é uma unidade atômica de informação, sendo expressa como um par chave-valor. Supercoluna é o agrupamento de colunas que podem ser obtidas juntas ou que possuem uma associação semântica entre si. Família de colunas é o agrupamento de colunas e de supercolunas que armazenam o mesmo tipo de dados, sendo a semelhança mais próxima da tabela no modelo relacional (INDRAWAN-SANTIAGO, 2012).

A estrutura de um BD colunar equivale ao esquema do banco de dados. No entanto, esse esquema é mais flexível — se comparado ao esquema de um BD relacional

¹⁰ Coleção de dados binários armazenados como uma única entidade.

¹¹ <http://basho.com/products/>

¹² <https://redis.io/>

¹³ <http://memcachedb.org/>

¹⁴ <https://www.oracle.com/database/berkeley-db/db.html>

¹⁵ <https://aws.amazon.com/pt/dynamodb/>

¹⁶ <http://www.project-voldemort.com/voldemort/>

— quanto à adição de uma nova coluna ou de uma nova supercoluna.

BDs populares nesta categoria são: *HBase*¹⁷, *Hypertable*¹⁸, *Google BigTable*¹⁹, *Cassandra*²⁰.

2.3.5 Banco de dados orientado a grafos

Um BD orientado a grafos é um BD que utiliza grafos como o meio para representar seu esquema. Seus componentes básicos são: os nós (vértices), os relacionamentos (arestas) e as propriedades (atributos) dos nós e dos relacionamentos (JMTAURO; S; A.B, 2012). Neste caso, o BD pode ser visto como um multigrafo rotulado e direcionado, em que cada par de nós pode ser conectado por mais de uma aresta.

Os BDs orientados a grafos são projetados para dados altamente complexos e conectados, que ultrapassam as capacidades de relacionamentos com *JOINS* dos BDs relacionais. São frequentemente bons para encontrar pontos comuns e anomalias em grandes conjuntos de dados. Permitem descobrir padrões que são difíceis de detectar ao usar representações tradicionais, como tabelas.

Os principais BDs orientados a grafos são: *Neo4J*²¹, *Infinite Graph*²² e *OrientDB*²³.

2.3.6 Banco de dados orientados a documentos

Os BDs orientados a documentos não estão preocupados com o alto desempenho em operações de leitura e escrita concorrentes, mas sim, em garantir um armazenamento eficiente de grandes volumes de dados e um bom desempenho em operações de consulta (HAN et al., 2011).

Esses BDs podem ser considerados como uma subcategoria dos BDs chave-valor. Sendo a diferença entre esses dois, o fato de que, enquanto os BDs chave-valor armazenam valores como BLOB, os BDs orientados a documentos armazenam os valores

¹⁷ <http://hbase.apache.org/>

¹⁸ <http://www.hypertable.org/>

¹⁹ <https://cloud.google.com/bigtable/>

²⁰ <https://cassandra.apache.org/>

²¹ <https://neo4j.com/>

²² <http://www.objectivity.com/products/infinitegraph/>

²³ <http://orientdb.com/>

como documentos, os quais podem ser armazenados em um formato de troca de dados padrão, como XML, JSON ou BSON ([HASHEM; RANC, 2016](#)).

Em geral, BDs orientados a documentos não possuem esquema, uma vez que o número de campos não é limitado e novos campos podem ser adicionados dinamicamente a um documento. Um documento, por sua vez, pode ser representado como uma estrutura de dados em forma de árvore, constituída de valores escalares (*string*, *boolean*, numérico), de documentos aninhados e coleções.

Como outros BDs NoSQL, BDs orientados a documentos não fornecem propriedades de transação ACID. BDs populares nessa categoria são o *MongoDB* e o *CouchDB*.

2.3.6.1 *MongoDB*

MongoDB é um sistema de gerenciamento de banco de dados NoSQL orientado a documentos, lançado em 2009. O *MongoDB* contém um conjunto de coleções, das quais não possuem esquemas de tabelas predefinidas. Os dados são armazenados como documentos BSON (objetos JSON codificados em binário) ([GYORODI et al., 2015](#)), e suas principais características são: flexibilidade, poder, velocidade e facilidade de uso. O *MongoDB* suporta servidores replicados e indexação, além de oferecer *drivers* para várias linguagens de programação ([BOICEA; RADULESCU; AGAPIN, 2012](#)).

2.4 JSON

JSON (*JavaScript Object Notation*) é um formato leve utilizado para intercâmbio de dados complexos e heterogêneos. Por ser um formato de texto completamente independente de linguagem ([ECMA-404, 2014](#)), utilizando convenções que são familiares aquelas utilizadas em linguagens como C, C++, C#, *Java*, *JavaScript*, *Perl* e *Python*, o formato JSON foi amplamente adotado como um formato ideal para intercâmbio de dados.

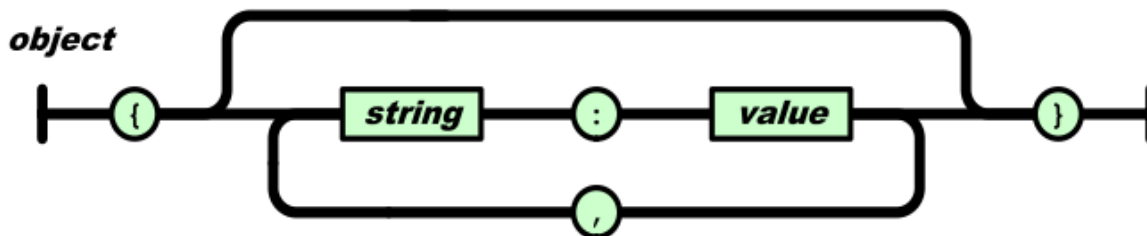
JSON é construído basicamente em duas estruturas, presentes em muitas linguagens de programação:

- Uma coleção de pares chave-valor, semelhante a um objeto.
- Uma lista ordenada de valores, semelhante a um *array*.

A terminologia que define a estrutura de um documento JSON compreende os seguintes conceitos:

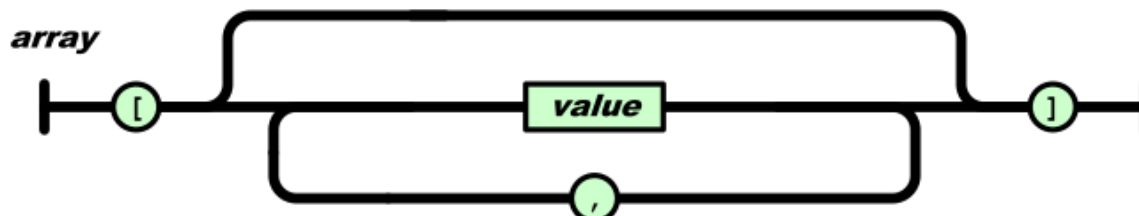
- *Objeto*: conjunto desordenado de pares chave-valor. Um objeto, representado na figura 4, começa com '{' (abre-chaves) e termina com '}' (fecha-chaves), cada chave é seguida de ':' (dois-pontos) e os pares chave-valor são separados por ',' (vírgula);
- *Array*: coleção ordenada de valores. Um *array*, representado na figura 5, começa com '[' (abre-colchetes) e termina com ']' (fecha-colchetes) e os valores são separados por ',' (vírgula);
- *Valor*: um valor JSON pode ser um valor de tipo primitivo (*string*, numérico, booleano (*true*, *false*), de tipo estruturado (objeto, *array*), ou nulo (*null*), como representado na figura 6. O uso de objetos e *arrays* como valores permite definir estruturas aninhadas.

Figura 4: Representação de um objeto em JSON



Fonte: [ECMA-404 \(2014\)](#)

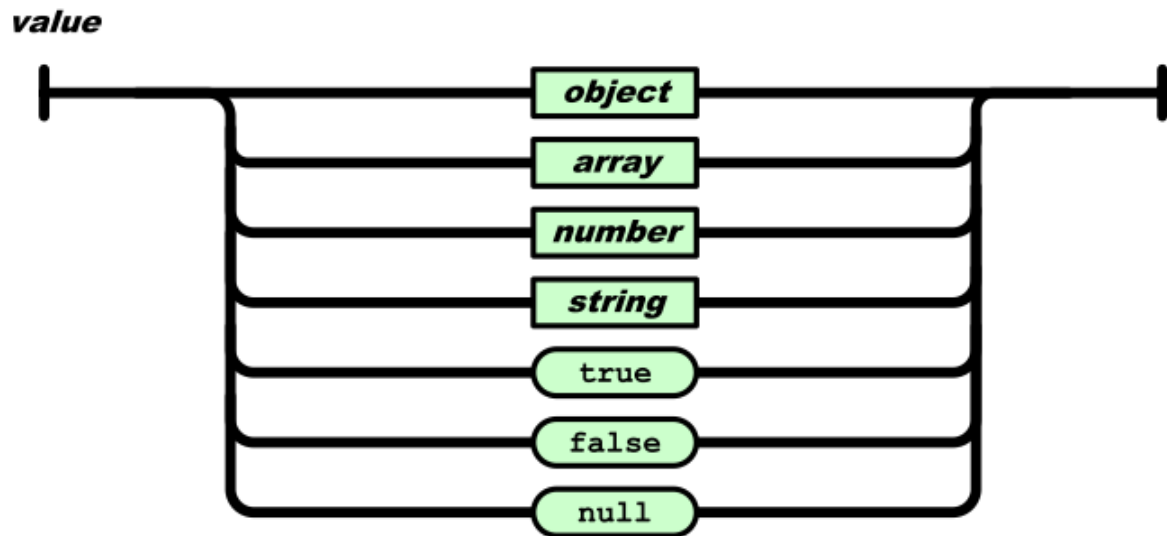
Figura 5: Representação de um *array* em JSON



Fonte: [ECMA-404 \(2014\)](#)

Por ser facilmente compreendido por desenvolvedores e máquinas, o JSON tornou-se o formato mais popular para enviar requisições às APIs (*Application Program-*

Figura 6: Representação de um valor em JSON

Fonte: [ECMA-404 \(2014\)](#)

ming Interface) e respostas sobre o protocolo HTTP (*Hypertext Transfer Protocol*), desempenhando assim um papel fundamental no contexto das aplicações *web* ([PEZOA et al., 2016](#)).

2.5 JSON Schema

Com a popularidade do formato JSON nas aplicações *web*, nota-se que em muitos cenários, uma forma declarativa de se especificar um esquema para documentos JSON pode ser algo benéfico. Com a definição de um esquema, pode-se evitar que uma API receba chamadas mal formadas, que podem ocasionar erros internos, caso alguma informação do documento não seja validada pela API ([PEZOA et al., 2016](#)).

A definição de um esquema também evita que a validação de um documento JSON seja feita durante a codificação da API, pois o esquema definido vai servir como um filtro que aceita somente documentos que estão no formato correto. Essa definição de esquema também serve como um meio de se padronizar a especificação de quais tipos de documento JSON são aceitos como entrada e saída de uma API.

O JSON Schema ²⁴ é uma tentativa de fornecer uma linguagem de esquema de propósito geral para JSON, que permita aos usuários restringir a estrutura dos docu-

²⁴ <http://json-schema.org/>

mentos JSON – nomes necessários em objetos, os tipos permitidos para um atributo de um objeto ou para valores em um *array* - além de oferecer uma estrutura para verificar a integridade das solicitações e sua conformidade com a API (PEZOA et al., 2016). Além disso, o JSON *Schema* permite que sejam incorporadas descrições nas especificações do esquema, permitindo que o autor do esquema forneça comentários em linha.

Há outras alternativas para definir esquemas para documentos JSON, mas estas são baseadas no próprio JSON *Schema* ou são projetadas com um determinado conjunto de casos de uso em mente. Alguns exemplos são: Swagger ²⁵, RAML ²⁶ e Google discovery ²⁷;

²⁵ <https://swagger.io/>

²⁶ <https://raml.org/>

²⁷ <https://developers.google.com/discovery/>

3 TRABALHOS RELACIONADOS

Este capítulo apresenta e compara as propostas de [Klettke, Störl e Scherzinger \(2015\)](#), [Ruiz, Morales e Molina \(2015\)](#) e [Wang et al. \(2015\)](#) para extração de esquemas de dados JSON, armazenados em bancos de dados NoSQL. Estas propostas serviram de base para este trabalho e, ao final, são comparadas com o presente trabalho.

3.1 *Schema Extraction and Structural Outlier Detection for JSON-based NoSQL Data Stores*

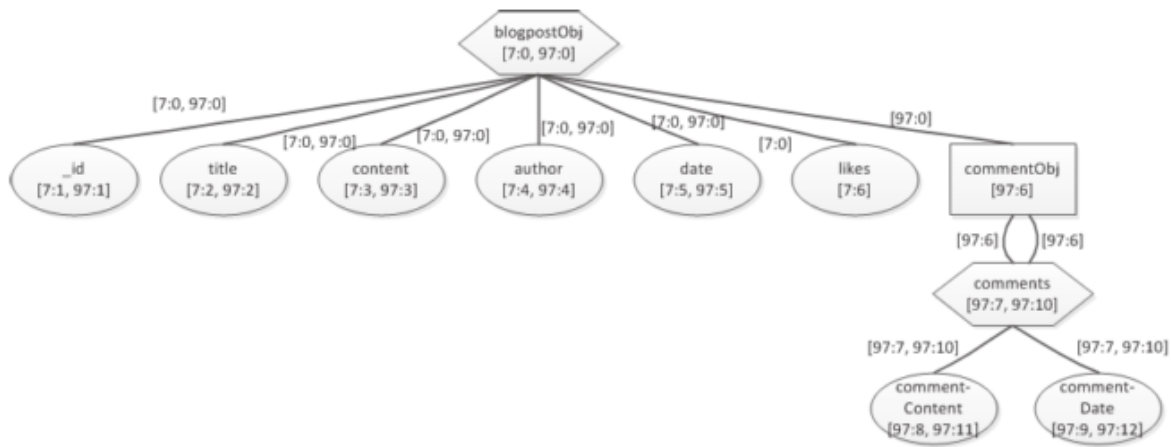
Os autores propõem um algoritmo para extração de esquemas de banco de dados NoSQL orientados a documentos *MongoDB*. Baseiam-se em ideias propostas para extrair DTDs (*Document Type Definition*) de coleções de documentos XML, mas levam em conta as particularidades dos dados JSON, como a natureza desordenada de seus elementos (ver seção 2.4), além de medidas de similaridade que descrevem a heterogeneidade estrutural dos mesmos.

O algoritmo proposto utiliza uma estrutura de dados em grafo direcionado, representado na figura 7, que resume a informação estrutural de todos os documentos analisados. Essa estrutura é composta por nós e arestas e permite detectar *outliers* estruturais — padrões que ocorrem em poucos conjuntos de dados —, além de determinar o grau de cobertura dos documentos, capturando a homogeneidade estrutural de uma coleção de documentos.

No contexto deste trabalho, os nós representam as propriedades de um objeto JSON (ver seção 2.4) e as arestas capturam a estrutura hierárquica dos documentos. Ambos são marcados com informações de linhagem, isto é, listas que especificam em que documentos JSON a propriedade estrutural está presente.

Para declarar um esquema e validar dados dado um esquema, é necessário definir uma linguagem de descrição de esquema ou utilizar alguma que já foi definida. No caso deste artigo, a linguagem de definição de esquema utilizada foi o JSON *Schema* (ver seção 2.5).

Figura 7: Grafo de Identificação de Estrutura proposto por Klettke, Störl e Scherzinger (2015)



Fonte: Klettke, Störl e Scherzinger (2015, p. 433)

O algoritmo é composto por quatro etapas: a) Seleção de documento, b) Extração da estrutura dos documentos JSON, c) Construção do Grafo de Identificação de Estrutura (SG) e d) Geração do JSON *Schema*, conforme representado na figura 8.

Na primeira etapa, a seleção dos documentos pode ser feita selecionando a coleção completa ou selecionando grupos de documentos JSON (distinguidos por *data*, *timestamp*, número de versão). No último caso, pode ser interessante para revelar a evolução do esquema ao longo do tempo, segundo os autores.

Na segunda etapa, é extraída a estrutura dos documentos JSONs e a estrutura do SG é definida. O SG é composto por um conjunto finito de nodos e de arestas ou arcos. Os nodos possuem informações como o nome da propriedade que o nodo representa, os documentos em que o nodo ocorre, bem como o caminho do nodo raiz até o respectivo nodo. Além dessas informações, um nodo contém uma ou mais arestas que direcionam para os nodos filhos ou nodos folhas.

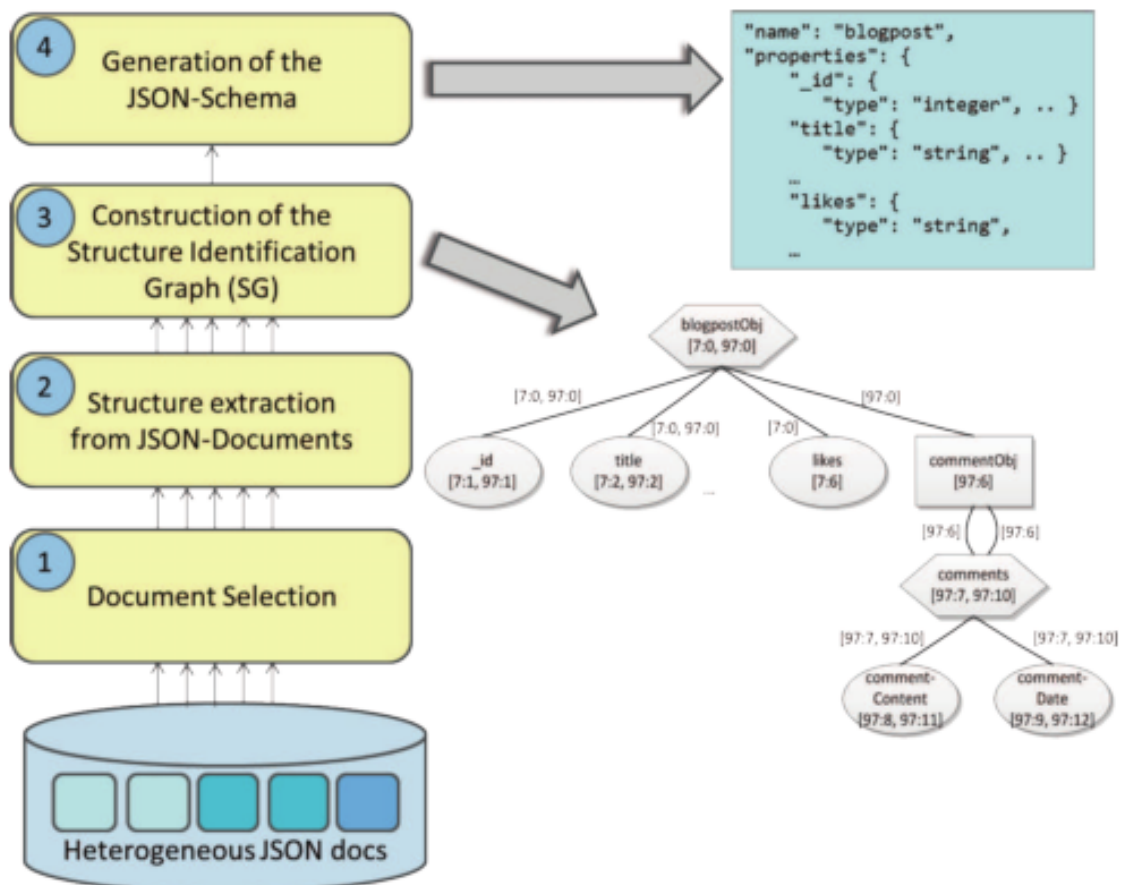
Na terceira etapa, é realizada a construção do SG. Primeiro é construído o SG a partir dos dados de entrada do banco de dados NoSQL. Os nodos no documento JSON são percorridos em *preorder*. Para cada nodo dos dados de entrada, é realizada a adição ou extensão de um nodo no SG e uma aresta ligando ao nodo pai é adicionada ou estendida.

A adição de um nodo no SG é distinguida em dois casos: se o nodo ainda não existe, então o nodo é adicionado, caso contrário, as informações do nodo atual são

anexadas à lista do nodo que já está no SG. A adição de uma aresta é semelhante. Se a aresta ainda não existe no SG, então ela é adicionada, caso contrário, a lista de arestas do nodo pai é atualizada.

Na quarta etapa, o JSON *Schema* é derivado do SG. Nesta etapa é verificado quais propriedades são obrigatórias e quais são opcionais, bem como o tipo de dados das propriedades. Para uma propriedade ser considerada obrigatória, ela deve aparecer em todos os documentos em que o nodo pai ocorre. Caso contrário ela é considerada opcional. Uma propriedade pode ter mais de um tipo de dado diferente, neste caso um tipo de dados de união é utilizado, como por exemplo: `"oneOf": [{ "type": "string" }, { "type": "integer" } ]`.

Figura 8: Etapas da Extração de esquemas proposta por Klettke, Störl e Scherzinger (2015)



Fonte: Klettke, Störl e Scherzinger (2015, p.430)

3.2 Inferring Versioned Schemas from NoSQL Databases and its Applications

Os autores propõem um processo de engenharia reversa para extração de esquemas versionados de banco de dados NoSQL orientados a agregados — chave-valor, documento e colunar — através da metodologia MDE (*Model Driven Engineering*).

No contexto desse trabalho, o banco de dados NoSQL é considerado como um *array* arbitrariamente grande de objetos JSON, representado na figura 9.

Figura 9: Exemplo de uma coleção de objetos JSON

```
{ ...
  "rows": [
    {
      "content": {
        "chapters": 33,
        "pages": 527
      },
      "authors": [
        {
          "company": {
            "country": "USA",
            "name": "IBM"
          },
          "name": "Grady Booch",
          "_id": "210"
        },
        {
          "company": {
            "country": "USA",
            "name": "IBM"
          },
          "name": "James Rumbaugh",
          "_id": "310"
        },
        {
          "country": "USA",
          "company": "Ivar Jacobson Consulting (IJC)",
          "name": "Ivar Jacobson",
          "_id": "410"
        }
      ],
      "type": "book",
      "year": 2013,
      "publisher_id": "345679",
      "title": "The Unified Modeling Language",
      "_id": "1"
    },
    {
      "discipline": "software engineering",
      "issn": [
        "0098-5589",
        "1939-3520"
      ],
      "name": "IEEE Trans. on Software Engineering",
      "type": "journal",
      "_id": "11"
    },
    {
      "name": "Automated Software Engineering",
      "issn": [
        "0928-8910",
        "1573-7535"
      ],
      "discipline": "software engineering",
      "type": "journal",
      "_id": "12",
      "number": 10515
    },
    {
      "city": "Barcelona",
      "name": "Omega",
      "type": "publisher",
      "_id": "123451"
    },
    {
      "city": "Newton",
      "name": "O'Reilly Media",
      "type": "publisher",
      "_id": "928672"
    },
    {
      "type": "book",
      "author": {
        "_id": "101",
        "name": "Bradley Holt",
        "company": {
          "country": "USA",
          "name": "IBM Cloudant",
        }
      },
      "title": "Writing and Querying MapReduce Views in CouchDB",
      "publisher_id": "928672",
      "_id": "2"
    },
    {
      "name": "Addison-Wesley",
      "type": "publisher",
      "_id": "345679"
    },
    {
      "type": "publisher",
      "journals": [
        "11",
        "12"
      ],
      "name": "IEEE Publications",
      "_id": "907863"
    }
  ]
}
```

Fonte: Ruiz, Morales e Molina (2015, p. 3)

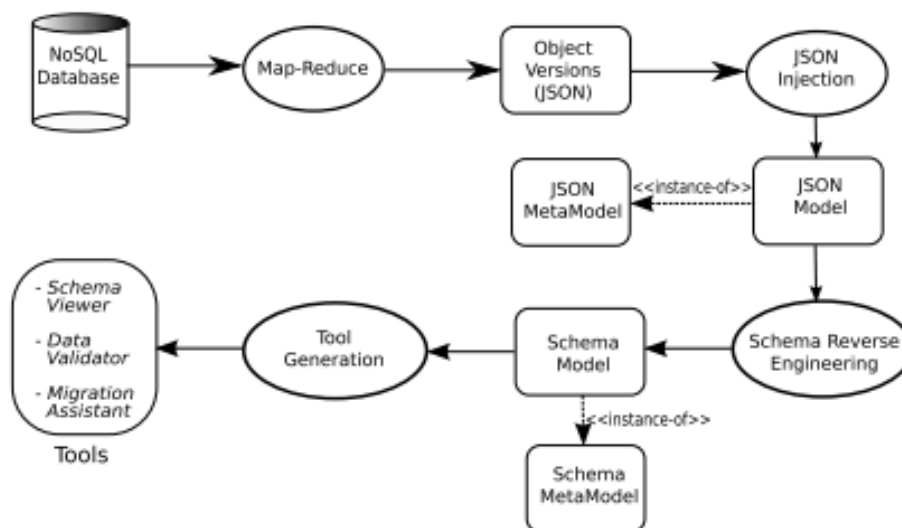
Um objeto JSON, por sua vez, deve seguir um formato que seja independente de sistema de banco de dados NoSQL, pois a proposta baseia-se em banco de dados NoSQL orientados a agregados. Sendo assim, o objeto deve possuir, no mínimo, dois

campos padrões:

- *type*: campo que descreve o tipo da entidade.
- *_id*: campo que representa um identificador exclusivo para o objeto.

O processo de extração de esquemas é composto por três etapas: a) Extração de objetos JSON, b) Definição do esquema JSON e c) Definição do esquema NoSQL, conforme representado na figura 10.

Figura 10: Etapas da Extração de esquemas proposta por Ruiz, Morales e Molina (2015)



Fonte: Ruiz, Morales e Molina (2015, p.6)

Na primeira etapa, é realizada uma operação de *map-reduce* para extrair uma coleção de objetos JSON, contendo um objeto para cada versão de uma entidade. Para cada objeto, a operação *map* executa um processo de duas etapas:

1. Gerar o identificador de versão: uma *string* obtida a partir da concatenação do valor do campo especial *type* com uma representação textual do esquema bruto do objeto. A representação do esquema bruto do objeto (*raw schema*), representado na figura 11, por sua vez, é um objeto JSON construído seguindo duas regras:

- a) ter a mesma estrutura que o objeto descrito com relação a campos, objetos aninhados e *arrays*;

- b) cada valor primitivo no objeto descrito é substituído no esquema bruto por seu tipo JSON (por exemplo, *string* ou *number*).

2. Gerar o par de chave-valor <identificador de versão, objeto>

Em seguida, a operação *reduce* seleciona um objeto para cada identificador de versão e adiciona na coleção de objetos JSON que é retornada.

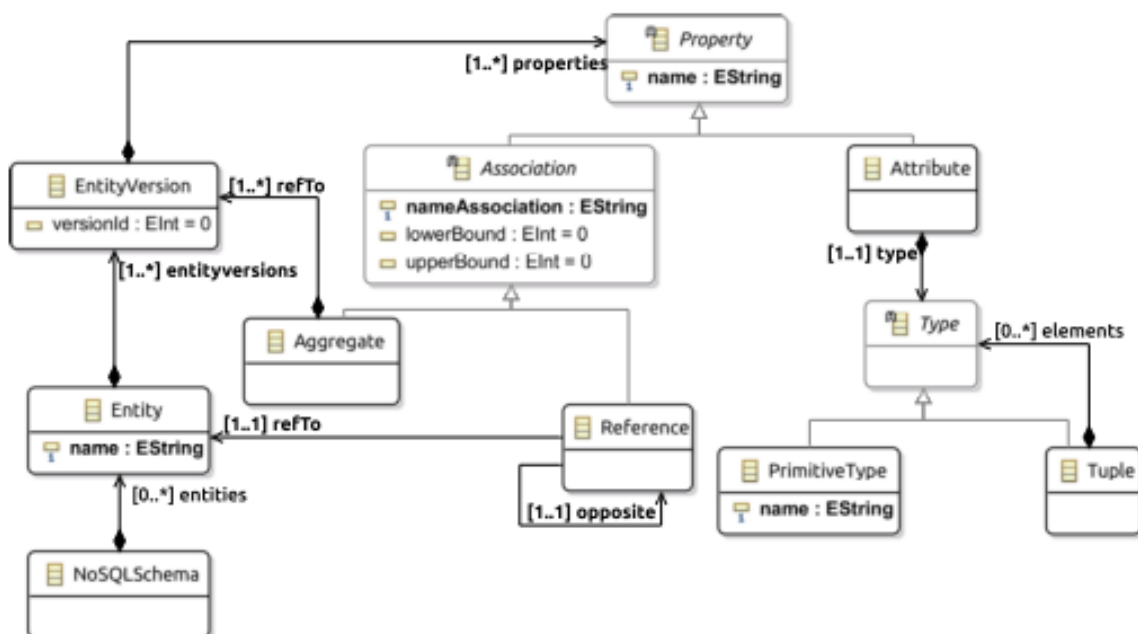
Na segunda etapa, a coleção de objetos JSON obtida na primeira etapa é injetada em um modelo JSON que está em conformidade com um metamodelo JSON.

Figura 11: Exemplo de esquema bruto (*raw schema*)

JSON object	Raw Schema
<code>{name:"Omega", city:"Barcelona"}</code>	<code>{name:String, city:String}</code>
<code>{title:"Writing and...", publisher_id:"928672", author:{name:"Bradley Holt", company:{country:"USA", name:"IBM Cloudant"}}}</code>	<code>{title:String, publisher_id:String, author:{name:String, company:{country:String, name:String}}}</code>

Fonte: Ruiz, Morales e Molina (2015, p.7)

Figura 12: Metamodelo NoSQL *Schema* para representar um esquema de banco de dados NoSQL orientado a agregados

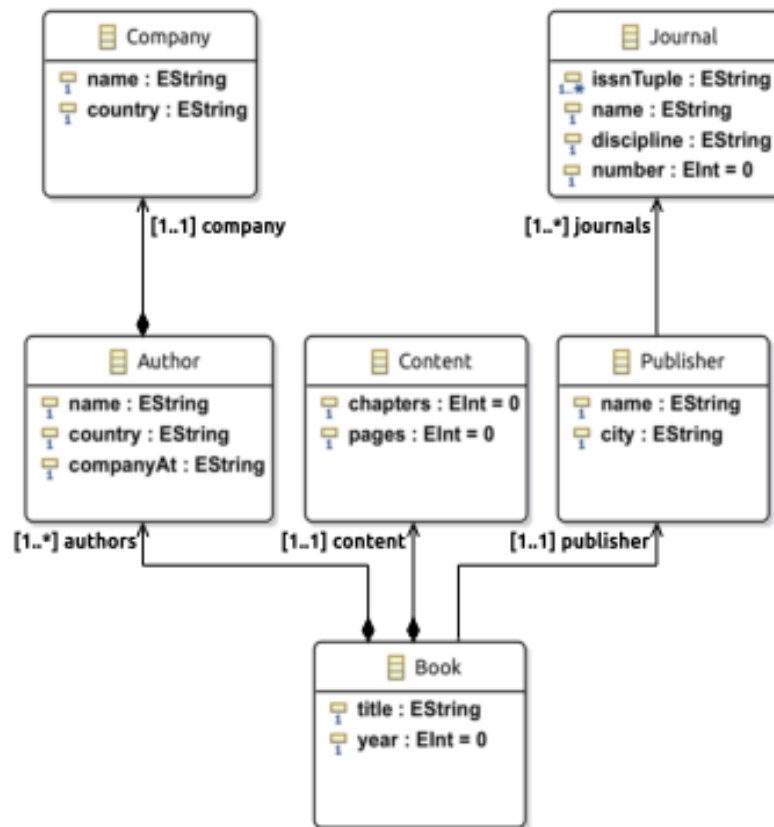


Fonte: Ruiz, Morales e Molina (2015, p.6)

Na terceira etapa, o processo de engenharia reversa é implementado como uma transformação de modelo para modelo (*model-to-model*), cuja entrada é o modelo JSON, gerado na segunda etapa, e a saída é um modelo que está em conformidade com o metamodelo NoSQL *Schema*, representado na figura 12. O processo de transformação de modelos, descobre os elementos do esquema: entidades e versões de entidades, atributos, relacionamentos por agregação e por referência.

A figura 13 representa um esquema NoSQL gerado para a coleção de objetos JSONs da figura 9. A partir do metamodelo NoSQL *Schema*, transformações podem ser aplicadas para gerar o esquema em qualquer formato, por exemplo, como um diagrama de classes UML (*Unified Modeling Language*).

Figura 13: Exemplo de um esquema NoSQL



Fonte: Ruiz, Morales e Molina (2015, p.10)

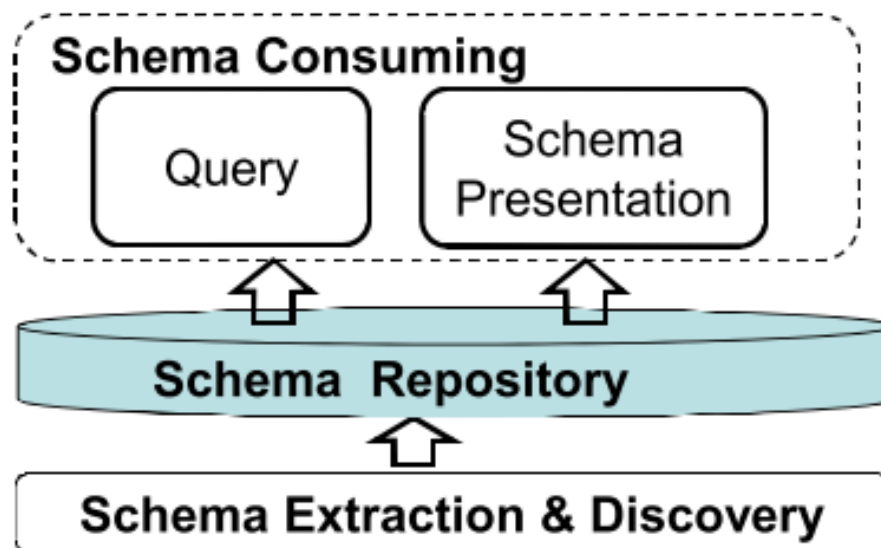
Os modelos de esquema NoSQL inferidos podem ser usados para construir utilitários de bancos de dados, que exigem conhecimento da estrutura do BD, como, por exemplo, SQL *query engines*, e ferramentas auxiliares, como validadores de dados, *scripts*

de migração ou diagramas de esquemas.

3.3 Schema Management for Document Stores

Os autores propõem um *framework*²⁸ para gerenciamento de esquemas de banco de dados NoSQL orientados a documento, representado na figura 14. O *framework* descobre e persiste esquemas de documentos JSON, além de suportar consultas e sumarização de esquemas.

Figura 14: *Framework* para gerenciamento de esquemas.



Fonte: Wang et al. (2015, p. 923)

O primeiro desafio na concepção de um *framework* de gerenciamento de esquemas para os bancos de dados NoSQL orientados a documentos, segundo os autores, é como descobrir eficientemente todos os esquemas distintos que aparecem em todos os registros, pois, normalmente, esses registros não possuem uma estrutura em comum, resultando assim, em um processo complexo de descoberta. Para enfrentar esse desafio, os autores propõem uma nova estrutura de dados, chamada de *eSiBu-Tree*, para a descoberta e armazenamento de esquemas.

O segundo desafio que os autores enfrentaram, foi referente à implementação de uma interface de consulta (semelhantes àquelas presentes em SGBDRs (Sistema de

²⁸ Um *framework*, em desenvolvimento de *software*, é uma abstração que une códigos comuns entre vários projetos de *software* provendo uma funcionalidade genérica (Wikimedia, 2017).

Gerenciamento de Banco de Dados Relacional) sobre o repositório de esquemas. Característica essencial para um *framework* de gerenciamento de esquemas.

O terceiro desafio, segundo os autores, se refere em como representar esquemas variados como um único esquema em cenários mais avançados de exploração de dados. Os autores apresentam um exemplo de uma coleção de dados da *DBPedia*²⁹. Neste exemplo, a interseção de todos os esquemas resulta em apenas um atributo, enquanto que a união resulta em 1.648 atributos, entre os quais, mais da metade aparecem apenas uma vez em todos os registros. Sendo assim, soluções simples como usar a interseção ou união de todos os esquemas não funcionam bem na prática. Para resolver esse problema, os autores propõem o conceito de "*skeleton*" para a visualização de esquemas, o qual representa o menor conjunto de atributos que melhor definem o núcleo do esquema.

O *framework* proposto possui três componentes, como mostrado na figura 14: componente de extração e descoberta de esquemas, componente de armazenamento de esquema e um componente para consulta e sumarização de esquemas.

Documentos JSON são obtidos a partir de um *dataset* JSON e, para cada documento, um *record schema* é extraído por meio da representação da estrutura do documento na *eSiBu-Tree*. Apenas os nomes dos campos são considerados para montar a estrutura do documento na *eSiBu-Tree* (não fazendo referência a tipos de dados). O *skeleton* é usado para gerar um esquema único (*core schema*) do objeto, o qual é formado pelos atributos que aparecem com maior frequência nos esquemas dos registros.

A tabela 1 apresenta um comparativo das principais características observadas nos trabalhos relacionados. As características consideradas são as seguintes: (i) Origem (fonte de origem usada para obter documentos JSON); (ii) Objetivo (perspectiva de uso dos esquemas extraídos); (iii) Abordagem (abordagem utilizada para obter o esquema final); (iv) Modelo Intermediário; (v) Modelo de saída (como o esquema é disponibilizado ao final do processo); (vi) Formato de entrada suportado (se suporta mais de um formato de entrada); (vii) Interface (se possui interface gráfica ou por linha de comando); e (viii) Etapas do processo (um resumo das etapas do processo de extração adotado).

²⁹ <http://wiki.dbpedia.org/>

Tabela 1: Comparação entre propostas para extração de esquemas de banco de dados NoSQL

	(KLETTKE; STÖRL; SCHERZINGER, 2015)	(RUIZ; MORALES; MOLINA, 2015)	(WANG et al., 2015)	(COSTA, 2017)
ORIGEM	<i>Dataset no MongoDB</i>	<i>MongoDB, CouchDB e HBase</i>	<i>Datasets JSON</i>	<i>Dataset no MongoDB</i>
OBJETIVO	Ferramenta para visualização e validação de esquemas.	Ferramenta para visualização e validação de esquemas	<i>Framework</i> para consulta e visualização de esquemas	Ferramenta <i>web</i> para extração de esquemas.
ABORDAGEM	Organização hierárquica (grafo)	Transformação de modelos	Organização hierárquica (arvore)	Organização hierárquica (grafo)
MOD. INTER-MEDIÁRIO	<i>Structure Identification Graph (SG)</i>	Metamodelo JSON	<i>eSiBu-Tree</i>	<i>Reduced Structure Identification Graph (RG)</i>
MOD. DE SAÍDA	JSON <i>Schema</i>	Metamodelo de esquema JSON	<i>eSiBu-Tree</i>	JSON <i>Schema</i>
FORMATO DE ENTRADA SUPORTADO	JSON	JSON	JSON	JSON e JSON Estendido
INTERFACE	CLI	CLI	CLI	GUI
ETAPAS DO PROCESSO	a) Seleção de documentos JSON; b) Extração da estrutura dos documentos; c) Construção do grafo SG; d) Geração do JSON <i>Schema</i>	a) Extração da estrutura dos dados JSON (<i>raw schema</i>) ; b) Transformação para esquemas JSON; c) Transformação para esquema NoSQL	a) Seleção de documentos JSON; b) Criação de registro na <i>eSiBu-Tree</i> ; c) Visualização do esquema unificado (<i>skeleton</i>)	a) Seleção de documentos JSON; b) Extração da estrutura dos documentos (<i>raw schema</i>); c) Construção do grafo RG; d) Geração do JSON <i>Schema</i>

Fonte: Autor

Este capítulo apresentou uma análise das propostas similares à ferramenta proposta neste trabalho. Após o estudo individual das propostas, foi realizado uma comparação entre as mesmas e a ferramenta proposta. Destaque para formato de entrada suportado e interface, características que justificam a implementação da ferramenta proposta neste trabalho de conclusão de curso.

4 ANDAMENTO DO TRABALHO

O presente TCC tem como objetivo desenvolver uma ferramenta *web* para extração de esquemas de documentos JSON em bancos de dados NoSQL orientados a documentos. Os diferenciais dessa ferramenta quanto aos trabalhos relacionados são:

- Interface GUI;
- Acesso via qualquer *browser*;
- Persistência dos esquemas gerados;
- Histórico de JSON *Schema* gerados;
- Visualização do JSON *Schema*;
- Suporte a JSON estendido, como *data*, *timestamp*, *objectId* etc.).

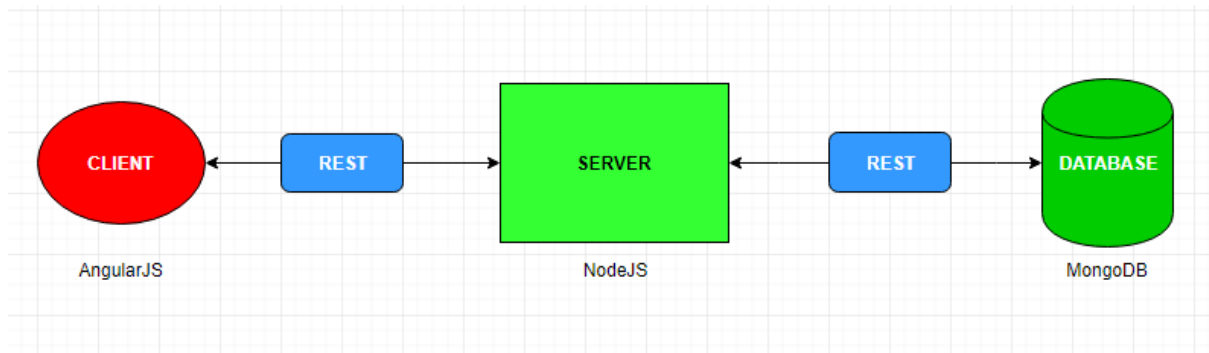
A primeira parte do trabalho focou principalmente na análise dos assuntos pertinentes ao desenvolvimento da ferramenta, bem como nos trabalhos relacionados, para levantamento de requisitos e características mínimas para a ferramenta. Ao final da primeira etapa do trabalho, era previsto que os capítulos de introdução, fundamentação teórica e de trabalhos relacionados estivessem concluídos.

A segunda parte do trabalho focará o desenvolvimento da ferramenta.

4.1 Arquitetura

A arquitetura utilizada é inspirada no modelo *client-server* e utiliza o padrão de projetos MVC (*Model*, *View*, *Controller*). O servidor será desenvolvido em *NodeJS* no esquema de *Web Service* REST. Ele será responsável por todo o controle da lógica da ferramenta. Para persistência de dados será utilizado o banco de dados NoSQL orientado a documentos *MongoDB*. O cliente será desenvolvido em HTML e *JavaScript*. Para renderização do conteúdo será utilizado o *framework AngularJS*, que é um *framework* MVC. A Figura 15 apresenta uma visão geral da arquitetura da ferramenta proposta. Toda a comunicação entre os componentes se dá por meio do protocolo REST.

Figura 15: Arquitetura da ferramenta proposta



Fonte: Autor

4.2 Processo de extração de esquemas

Das etapas do processo de extração de esquemas, as seguintes atividades se encontram prontas e foram desenvolvidas no *backend* da ferramenta:

- Seleção de documentos JSON;
- Extração da estrutura (*raw esquema*) dos documentos.

Na primeira etapa do processo de extração de esquemas, são selecionados todos os documentos JSON de uma coleção do *MongoDB*. A figura 16 mostra um exemplo de um documento JSON de um sistema de cadastro de usuários. As propriedades *address*, *emails*, *telephones* e *web_sites* representam *arrays* de objetos. As propriedades *last_login* e *_id* representam tipos de dados presentes no JSON Estendido. A figura 17 mostra o respectivo documento JSON armazenado no banco de dados *MongoDB*.

Após selecionar a coleção de documentos, a estrutura dos mesmos é extraída. Neste processo, é utilizada uma abordagem de extração de estrutura semelhante à apresentada por Ruiz, Morales e Molina (2015). O *raw schema*, representado na figura 18, contém a mesma estrutura que o documento apresentado na figura 17 com relação a campos, objetos aninhados e *arrays*. Cada valor primitivo no objeto descrito é substituído no *raw schema* por seu tipo de dado JSON (por exemplo, *string* ou *number*). Um diferencial nesta proposta é o fato de considerar os tipos de dados presentes no JSON Estendido do *MongoDB*, como por exemplo: *date*, *objectId*, entre outros.

Figura 16: Exemplo de um documento JSON

```
[
  {
    "id": {
      "$oid": "50319412fe4dce143835c552"
    },
    "address": [
      {
        "country name": "Brasil",
        "extended address": null,
        "locality": "Florianópolis",
        "post office box": null,
        "postal code": "88888-888",
        "region": "SC",
        "street_address": "Rua Teste, 150"
      }
    ],
    "birth date": "08/11/1991",
    "emails": [
      {
        "address": "email1@mail.com",
        "type": "PERSONAL"
      },
      {
        "address": "email2@mail.com.br",
        "type": "PERSONAL"
      }
    ],
    "first name": "Felipe",
    "gender": "male",
    "job": "Software Developer.",
    "last login": {
      "$date": "2017-06-26T22:26:33.000Z"
    },
    "last name": "de Souza da Costa",
    "legal person": false,
    "membership status": "ACTIVE",
    "telephones": [
      {
        "ddd": 48,
        "ddi": 55,
        "number": "8888-8888",
        "type": "PERSONAL_CELLULAR"
      },
      {
        "ddd": 48,
        "ddi": 55,
        "number": "3333-3333",
        "type": "PERSONAL_PHONE"
      }
    ],
    "web_sites": []
  }
]
```

Fonte: Autor

Figura 17: Exemplo de um documento armazenado no *MongoDB*

Key	Value
(1) ObjectId("50319412fe4dce143835c552")	{ 13 fields }
_id	ObjectId("50319412fe4dce143835c552")
address	[1 element]
[0]	{ 7 fields }
country_name	Brasil
extended_address	null
locality	Florianópolis
post_office_box	null
postal_code	88888-888
region	SC
street_address	Rua Teste, 150
birth_date	08/11/1991
emails	[2 elements]
[0]	{ 2 fields }
address	email1@mail.com
type	PERSONAL
[1]	{ 2 fields }
address	email2@mail.com.br
type	PERSONAL
first_name	Felipe
gender	male
job	Software Developer.
last_login	2017-06-26 22:26:33.000Z
last_name	de Souza da Costa
legal_person	false
membership_status	ACTIVE
telephones	[2 elements]
[0]	{ 4 fields }
ddd	48
ddi	55
number	8888-8888
type	PERSONAL_CELLULAR
[1]	{ 4 fields }
ddd	48
ddi	55
number	3333-3333
type	PERSONAL_PHONE
web_sites	[0 elements]

Fonte: Autor

Figura 18: Exemplo do *Raw Schema* gerado

```
raw schemas [  
  {  
    "_id": "ObjectID",  
    "address": [  
      {  
        "country_name": "String",  
        "extended_address": "Null",  
        "locality": "String",  
        "post_office_box": "Null",  
        "postal_code": "String",  
        "region": "String",  
        "street_address": "String"  
      }  
    ],  
    "birth_date": "String",  
    "emails": [  
      {  
        "address": "String",  
        "type": "String"  
      },  
      {  
        "address": "String",  
        "type": "String"  
      }  
    ],  
    "first_name": "String",  
    "gender": "String",  
    "job": "String",  
    "last_login": "Date",  
    "last_name": "String",  
    "legal_person": "Boolean",  
    "membership_status": "String",  
    "telephones": [  
      {  
        "ddd": "Number",  
        "ddi": "Number",  
        "number": "String",  
        "type": "String"  
      },  
      {  
        "ddd": "Number",  
        "ddi": "Number",  
        "number": "String",  
        "type": "String"  
      }  
    ],  
    "web_sites": []  
  }  
]
```

Fonte: Autor

5 ATIVIDADES FUTURAS

Apesar dos resultados parciais terem sido satisfatórios e estarem dentro do cronograma estabelecido, a continuação do trabalho ainda demanda um grande esforço. Para a segunda etapa deve-se desenvolver:

Quanto ao processo de extração de esquema:

- Construção do grafo RG (estrutura intermediária);
- Geração do JSON *Schema*.

Quanto ao desenvolvimento da ferramenta:

- Finalização do *backend*;
- Desenvolvimento do *frontend*;
- Desenvolvimento do serviço de persistência;
- Integração do *frontend* com o *backend*;
- Realização de estudo de caso para validação da ferramenta.

Quanto a monografia:

- Concluir capítulo referente à ferramenta proposta;
- Apresentação dos resultados obtidos no estudo de caso;
- Revisão e finalização do documento.

REFERÊNCIAS

Big Data is the new black. *What is Big Data*. 2016. Disponível em: <<http://bigdata-black/featured/what-is-big-data/>>. Acesso em: 16 jun. 2017. Citado 2 vezes nas páginas 20 e 21.

BOICEA, A.; RADULESCU, F.; AGAPIN, L. I. MongoDB vs Oracle - Database comparison. *Proceedings - 3rd International Conference on Emerging Intelligent Data and Web Technologies, EIDWT 2012*, p. 330–335, 2012. Citado na página 27.

BREWER, E. A. Towards robust distributed systems (abstract). *Proceedings of the nineteenth annual ACM symposium on Principles of distributed computing - PODC '00*, v. 19, p. 7, 2000. ISSN 01635700. Disponível em: <<http://portal.acm.org/citation.cfm?doid=343477.343502>>. Citado 3 vezes nas páginas 21, 22 e 23.

BREWER, E. A. CAP twelve years later: How the "rules" have changed. *Computer*, v. 45, n. 2, p. 23–29, 2012. ISSN 0018-9162. Disponível em: <<http://ieeexplore.ieee.org/document/6133253/>>. Citado na página 22.

CATTELL, R. Scalable SQL and NoSQL data stores. *ACM SIGMOD Record*, v. 39, n. 4, p. 12, 2011. ISSN 01635808. Disponível em: <<http://portal.acm.org/citation.cfm?doid=1978915.1978919>>. Citado na página 13.

ECMA-404. The JavaScript Object Notation (JSON) Data Interchange Format. *ECMA International*, v. 1st Editio, n. October, p. 8, 2014. ISSN 2070-1721. Disponível em: <<https://www.rfc-editor.org/info/rfc7159>>. Citado 3 vezes nas páginas 27, 28 e 29.

GYORODI, C. et al. A comparative study: MongoDB vs. MySQL. *2015 13th International Conference on Engineering of Modern Electric Systems, EMES 2015*, p. 0–5, 2015. Citado na página 27.

HAN, J. et al. Survey on NoSQL database. *Proceedings - 2011 6th International Conference on Pervasive Computing and Applications, ICPCA 2011*, p. 363–366, 2011. ISSN 978-1-4577-0207-5. Citado 5 vezes nas páginas 13, 14, 19, 25 e 26.

HASHEM, H.; RANC, D. Evaluating NoSQL document oriented data model. *Proceedings - 2016 4th International Conference on Future Internet of Things and Cloud Workshops, W-FiCloud 2016*, p. 51–56, 2016. Disponível em: <<http://ieeexplore.ieee.org/document/7592700/>>. Citado 4 vezes nas páginas 14, 23, 24 e 27.

IETF. *RFC 7159 - The JavaScript Object Notation (JSON) Data Interchange Format*. 2014. Disponível em: <<https://tools.ietf.org/html/rfc7159>>. Acesso em: 29 jun. 2017. Citado na página 16.

INDRAWAN-SANTIAGO, M. Database research: Are we at a crossroad? Reflection on NoSQL. *Proceedings of the 2012 15th International Conference on Network-Based Information Systems, NBIS 2012*, p. 45–51, 2012. Citado 2 vezes nas páginas 14 e 25.

JMTAURO, C.; S, A.; A.B, S. Comparative Study of the New Generation, Agile, Scalable, High Performance NOSQL Databases. *International Journal of Computer Applications*, v. 48, n. 20, p. 1–4, 2012. ISSN 09758887. Disponível em: <<http://research-ijcaonline.org/volume48/number20/pxc3880336.pdf>>. Citado 2 vezes nas páginas 25 e 26.

JSON Schema. *Specification Links*. 2017. Disponível em: <<http://json-schema.org/specification-links.html>>. Acesso em: 29 jun. 2017. Citado na página 16.

KAISLER, S. et al. Big Data: Issues and Challenges Moving Forward. *2013 46th Hawaii International Conference on System Sciences*, p. 995–1004, 2013. ISSN 1530-1605. Disponível em: <<http://ieeexplore.ieee.org/document/6479953/>>. Citado na página 21.

KATAL, A.; WAZID, M.; GOUDAR, R. H. Big data: Issues, challenges, tools and Good practices. *2013 6th International Conference on Contemporary Computing, IC3 2013*, p. 404–409, 2013. Citado 2 vezes nas páginas 20 e 21.

KLETTKE, M.; STÖRL, U.; SCHERZINGER, S. Schema Extraction and Structural Outlier Detection for JSON-based NoSQL Data Stores. In: *Btw.* [S.l.]: GI, 2015. (LNI, v. 241), p. 425–444. ISBN 9783885796350. ISSN 16175468. Citado 6 vezes nas páginas 7, 15, 31, 32, 33 e 40.

PEZOA, F. et al. Foundations of JSON Schema. In: *Proceedings of the 25th International Conference on World Wide Web - WWW '16*. International World Wide Web Conferences Steering Committee, 2016. (WWW '16), p. 263–273. ISBN 9781450341431. Disponível em: <<http://dl.acm.org/citation.cfm?doid=2872427.2883029>>. Citado 2 vezes nas páginas 29 e 30.

RUIZ, D. S.; MORALES, S. F.; MOLINA, J. G. Inferring Versioned Schemas from NoSQL Databases and its Applications. *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, v. 9381, n. October, p. 467–480, 2015. ISSN 16113349. Disponível em: <http://link.springer.com/10.1007/978-3-319-25264-3_35>. Citado 10 vezes nas páginas 7, 13, 14, 31, 34, 35, 36, 37, 40 e 42.

SADALAGE, P. J.; FOWLER, M. *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. [S.l.: s.n.], 2012. 192 p. ISSN 1098-. ISBN 9780321826626. Citado 7 vezes nas páginas 13, 14, 15, 19, 23, 24 e 25.

SAGIROGLU, S.; SINANC, D. Big data: A review. *2013 International Conference on Collaboration Technologies and Systems (CTS)*, p. 42–47, 2013. ISSN 9781467364034. Disponível em: <<http://ieeexplore.ieee.org/document/6567202/>>. Citado na página 20.

TUDORICA, B. G.; BUCUR, C. A comparison between several NoSQL databases with comments and notes. *Proceedings - RoEduNet IEEE International Conference*, 2011. ISSN 20681038. Citado 2 vezes nas páginas 13 e 23.

WANG, L. et al. Schema management for document stores. *Proceedings of the VLDB Endowment*, v. 8, n. 9, p. 922–933, 2015. ISSN 21508097. Disponível em: <<http://dl.acm.org/citation.cfm?doid=2777598.2777601>>. Citado 3 vezes nas páginas 31, 38 e 40.

Wikimedia. *Software framework*. 2017. Disponível em: <https://en.wikipedia.org/wiki/Software_framework>. Acesso em: 29 jun. 2017. Citado na página 38.